

Báo Cáo Dự Án Phần Mềm (CNPM)

Tên Nhóm / Tên Sinh viên

Ngày 3 tháng 8 năm 2026

Mục lục

1	GitHub Connection & Repository Management	3
1.1	Introduction	3
1.2	GitHub Connection Using OAuth2	3
1.3	Repository Management	4
1.4	CI/CD Integration with GitHub Actions	5
1.5	Pull Request and Commit Tracking	7
1.6	Security and Error Handling	8
2	System Requirements	9
2.1	Requirements Introduction	9
2.2	Functional Requirements	10
2.2.1	FR1: GitHub Authentication Using OAuth2	10
2.2.2	FR2: Repository Selection and Management	10
2.2.3	FR3: Commit and Pull Request Synchronization	11
2.2.4	FR4: CI/CD Integration for Automatic Documenta- tion Update	11
2.2.5	FR5: Synchronization Status Management	11
2.3	Non-Functional Requirements	12
2.3.1	NFR1: Security	12
2.3.2	NFR2: Performance	12
2.3.3	NFR3: Scalability	13
2.3.4	NFR4: Reliability and Recovery	13
2.3.5	NFR5: Usability	13
2.4	System Constraints	13
2.4.1	Official GitHub API	13
2.4.2	Repository Access Scope	14
2.4.3	LivingDocs Internal API Integration	14
2.5	Conclusion	14

1 GitHub Connection & Repository Management

1.1 Introduction

GitHub được sử dụng như nền tảng quản lý mã nguồn của dự án LivingDocs. Toàn bộ tài liệu, mã nguồn và lịch sử thay đổi đều được lưu trữ tập trung trên GitHub nhằm hỗ trợ các thành viên làm việc nhóm hiệu quả hơn.

Trong dự án Công nghệ Phần mềm, việc tích hợp GitHub với hệ thống **LivingDocs** là cần thiết nhằm đạt được các mục tiêu sau:

- Quản lý mã nguồn tập trung và nhất quán giữa các thành viên trong nhóm.
- Đồng bộ thay đổi mã nguồn với hệ thống tài liệu tự động.
- Theo dõi lịch sử commit và Pull Request để kiểm soát tiến độ phát triển.
- Hỗ trợ quy trình review và kiểm tra chất lượng phần mềm trước khi triển khai.
- Giảm tình trạng tài liệu bị lỗi thời so với mã nguồn thực tế.

Nhờ khả năng tích hợp trực tiếp với GitHub, LivingDocs có thể tự động phát hiện thay đổi của mã nguồn và hỗ trợ cập nhật tài liệu tương ứng, giúp duy trì tính đồng bộ giữa tài liệu kỹ thuật và hệ thống đang phát triển.

1.2 GitHub Connection Using OAuth2

Để người dùng có thể kết nối tài khoản GitHub với LivingDocs mà không phải cung cấp mật khẩu trực tiếp, hệ thống sử dụng cơ chế OAuth2 Authorization Code Flow do GitHub hỗ trợ.

Cơ chế OAuth2 OAuth2 cho phép người dùng cấp quyền truy cập cho LivingDocs mà không cần chia sẻ mật khẩu GitHub. Hệ thống chỉ nhận được một `access token` với phạm vi quyền hạn đã được người dùng chấp thuận.

Quy trình đăng nhập và xác thực

1. Người dùng chọn chức năng **Connect GitHub**.
2. Hệ thống chuyển hướng đến trang xác thực của GitHub.

3. Người dùng đăng nhập GitHub và cấp quyền truy cập repository.
4. GitHub trả về một `authorization code`.
5. LivingDocs gửi code này đến GitHub để lấy `access token`.
6. Token được mã hóa và lưu trữ an toàn trong hệ thống.

Lợi ích của OAuth2

- **Bảo mật cao:** không lưu mật khẩu GitHub trong hệ thống.
- **Phân quyền rõ ràng:** chỉ truy cập những repository được cấp phép.
- **Dễ thu hồi quyền:** người dùng có thể hủy quyền truy cập bất kỳ lúc nào trên GitHub.
- **Tuân thủ chuẩn hiện đại:** tương thích với các chính sách bảo mật của GitHub.

Cơ chế này giúp LivingDocs duy trì khả năng tích hợp mạnh mẽ với GitHub mà vẫn đảm bảo an toàn cho dữ liệu và tài khoản của người dùng.

1.3 Repository Management

Sau khi kết nối GitHub thành công, LivingDocs cho phép người dùng lựa chọn các repository cần quản lý và đồng bộ với hệ thống tài liệu.

Lựa chọn Repository Hệ thống sử dụng GitHub REST API để lấy danh sách repository mà người dùng có quyền truy cập. Người dùng có thể chọn một hoặc nhiều repository để đưa vào workspace của LivingDocs.

Thông tin được lưu trữ bao gồm:

- Repository ID.
- Tên repository.
- Chủ sở hữu (owner).
- Nhánh mặc định (default branch).
- Thời gian đồng bộ gần nhất.

Đồng bộ dữ liệu từ Branch chính Theo thiết kế của hệ thống, sau khi kết nối GitHub thành công, người dùng sẽ có thể lựa chọn repository cần quản lý. Thông tin của repository sẽ được lưu để phục vụ cho quá trình theo dõi tài liệu và đồng bộ sau này.

- Cập nhật commit mới nhất.
- Lấy danh sách file thay đổi.
- Đồng bộ Pull Request đã merge.
- Cập nhật metadata của repository.

Theo dõi trạng thái cập nhật Hệ thống duy trì trạng thái đồng bộ thông qua các chỉ số sau:

- Commit cuối cùng đã xử lý.
- Pull Request đang mở.
- Pull Request đã merge.
- Thời gian đồng bộ gần nhất.
- Trạng thái thành công hoặc thất bại của lần đồng bộ.

Trong giai đoạn hiện tại của dự án, nội dung này mới dừng ở mức thiết kế và mô tả giải pháp. Việc triển khai chức năng kết nối và đồng bộ Repository sẽ được thực hiện sau khi các thành phần Backend và Database của hệ thống được hoàn thiện.

1.4 CI/CD Integration with GitHub Actions

Để hỗ trợ quá trình phát triển trong các giai đoạn tiếp theo, LivingDocs dự kiến tích hợp với **GitHub Actions** nhằm tự động hóa quy trình **Continuous Integration / Continuous Deployment (CI/CD)**. Việc tích hợp này sẽ được triển khai khi hệ thống hoàn thành các thành phần Backend, Frontend và Database.

Workflow tự động Build/Test Trong giai đoạn triển khai hệ thống, khi có commit mới được đẩy lên repository hoặc Pull Request được tạo, GitHub Actions có thể được cấu hình để tự động kích hoạt workflow kiểm tra. Quy trình thường bao gồm:

1. Checkout mã nguồn.
2. Thiết lập môi trường chạy.
3. Build project.
4. Chạy unit test và integration test.
5. Gửi kết quả về GitHub.

Tự động cập nhật tài liệu khi Merge PR Sau khi Pull Request được merge vào branch chính, workflow có thể gọi API của LivingDocs để:

- Phân tích thay đổi của mã nguồn.
- Xác định tài liệu liên quan cần cập nhật.
- Sinh đề xuất thay đổi tài liệu.
- Đồng bộ phiên bản tài liệu mới vào hệ thống.

Lợi ích của CI/CD

- **Giảm lỗi thủ công:** hạn chế quên cập nhật tài liệu hoặc bỏ sót bước kiểm tra.
- **Tăng tốc độ phát triển:** kiểm tra tự động giúp phát hiện lỗi sớm hơn.
- **Đảm bảo chất lượng:** mọi thay đổi đều được build và test trước khi đưa vào branch chính.
- **Duy trì tính đồng bộ:** tài liệu luôn được cập nhật theo thay đổi của mã nguồn.

Trong giai đoạn triển khai sau này, nhóm dự kiến tích hợp GitHub Actions để hỗ trợ quy trình CI/CD.

1.5 Pull Request and Commit Tracking

Theo dõi Pull Request và commit là chức năng quan trọng giúp LivingDocs liên kết sự thay đổi của mã nguồn với tài liệu kỹ thuật tương ứng.

Ghi nhận thay đổi từ Pull Request Khi GitHub gửi sự kiện `pull_request`, hệ thống sẽ:

1. Nhận webhook từ GitHub.
2. Xác minh chữ ký bảo mật của webhook.
3. Lấy danh sách file được thêm, sửa hoặc xóa.
4. Phân loại mức độ ảnh hưởng đến tài liệu.
5. Tạo bản ghi theo dõi trong hệ thống LivingDocs.

Liên kết Commit với tài liệu Mỗi commit được gắn với:

- Module hoặc thành phần hệ thống bị ảnh hưởng.
- API hoặc class được thay đổi.
- Tài liệu thiết kế hoặc tài liệu API liên quan.
- Phiên bản tài liệu cần cập nhật.

Cơ chế này cho phép truy vết từ tài liệu đến commit và ngược lại, hỗ trợ kiểm toán và bảo trì hệ thống hiệu quả hơn.

Quy trình Review và Merge

1. Developer tạo Pull Request.
2. GitHub Actions tự động build và chạy test.
3. LivingDocs phân tích ảnh hưởng đến tài liệu.
4. Reviewer kiểm tra mã nguồn và đề xuất cập nhật tài liệu.
5. Sau khi được chấp thuận, Pull Request được merge vào branch chính.
6. Hệ thống kích hoạt đồng bộ tài liệu cuối cùng.

Quy trình này đảm bảo mọi thay đổi quan trọng của mã nguồn đều được xem xét cùng với tài liệu tương ứng trước khi chính thức tích hợp vào hệ thống.

1.6 Security and Error Handling

Do GitHub Integration làm việc với dịch vụ bên ngoài và dữ liệu mã nguồn quan trọng, hệ thống cần có cơ chế bảo mật và xử lý lỗi phù hợp để đảm bảo tính ổn định và an toàn.

Token hết hạn Access token của GitHub có thể hết hạn hoặc bị người dùng thu hồi quyền truy cập. Khi phát hiện token không còn hợp lệ, LivingDocs sẽ:

- Đánh dấu trạng thái kết nối là **EXPIRED**.
- Tạm dừng quá trình đồng bộ repository.
- Hiện thị thông báo yêu cầu người dùng kết nối lại GitHub.

Lỗi kết nối GitHub Các lỗi phổ biến bao gồm:

- Mất kết nối mạng.
- GitHub API không phản hồi.
- Vượt quá giới hạn request (rate limit).
- Webhook gửi thất bại.

Trong trường hợp GitHub API không phản hồi, hệ thống sẽ thử gửi lại yêu cầu nhiều lần trước khi thông báo lỗi đến người dùng.

Cơ chế cảnh báo và Refresh Token LivingDocs hỗ trợ:

- Cảnh báo khi token sắp hết hạn.
- Gửi thông báo đến người dùng qua giao diện hệ thống.
- Nếu access token không còn hợp lệ hoặc người dùng thu hồi quyền truy cập, hệ thống sẽ yêu cầu người dùng thực hiện kết nối lại GitHub để tiếp tục đồng bộ dữ liệu.
- Ghi log toàn bộ quá trình xác thực và đồng bộ để hỗ trợ kiểm tra và khắc phục sự cố.

Đảm bảo an toàn dữ liệu khi đồng bộ Để bảo vệ dữ liệu mã nguồn và thông tin xác thực, hệ thống áp dụng các biện pháp sau:

- Token được lưu trữ dưới dạng mã hóa nhằm đảm bảo an toàn thông tin và hạn chế nguy cơ rò rỉ dữ liệu xác thực.
- Chỉ sử dụng giao thức HTTPS/TLS cho mọi kết nối với GitHub.
- Xác minh chữ ký `X-Hub-Signature-256` của webhook để chống giả mạo yêu cầu.
- Phân quyền truy cập theo vai trò người dùng (RBAC).
- Không ghi token hoặc dữ liệu nhạy cảm vào log ứng dụng.

Các nội dung trình bày trong chương này là cơ sở cho việc triển khai chức năng GitHub Integration trong các Sprint tiếp theo. Trong giai đoạn hiện tại, nhóm tập trung hoàn thiện yêu cầu, thiết kế hệ thống và tài liệu kỹ thuật trước khi bắt đầu phát triển mã nguồn.

2 System Requirements

2.1 Requirements Introduction

Tài liệu này mô tả các yêu cầu chức năng và phi chức năng của hệ thống **LivingDocs** trong phạm vi dự án Công nghệ Phần mềm. Mục tiêu của phần yêu cầu là xác định rõ những chức năng hệ thống cần cung cấp, các điều kiện hoạt động, cũng như các ràng buộc kỹ thuật cần tuân thủ trong quá trình thiết kế và triển khai.

Hệ thống LivingDocs được xây dựng nhằm hỗ trợ đồng bộ tài liệu kỹ thuật với mã nguồn được quản lý trên GitHub. Việc xác định đầy đủ yêu cầu giúp đảm bảo hệ thống đáp ứng đúng nhu cầu quản lý tài liệu, theo dõi thay đổi mã nguồn và hỗ trợ quy trình phát triển phần mềm hiện đại.

Phạm vi hệ thống Phạm vi của hệ thống bao gồm:

- Kết nối tài khoản GitHub thông qua cơ chế OAuth2.
- Quản lý repository cần theo dõi và đồng bộ.
- Theo dõi commit, branch và Pull Request.
- Đồng bộ tự động dữ liệu mã nguồn với hệ thống LivingDocs.

- Hỗ trợ tích hợp CI/CD để tự động cập nhật tài liệu khi có thay đổi mã nguồn.

Những yêu cầu này đóng vai trò nền tảng cho việc thiết kế kiến trúc hệ thống và triển khai các thành phần GitHub Integration của LivingDocs.

2.2 Functional Requirements

2.2.1 FR1: GitHub Authentication Using OAuth2

Hệ thống phải cho phép người dùng đăng nhập và kết nối tài khoản GitHub bằng giao thức **OAuth2**.

Mô tả

- Người dùng chọn chức năng **Connect GitHub**.
- Hệ thống chuyển hướng đến trang xác thực GitHub.
- Sau khi người dùng cấp quyền truy cập, hệ thống nhận được **authorization code**.
- Mã xác thực được trao đổi để lấy **access token**.
- Token được lưu trữ an toàn để sử dụng cho các yêu cầu API tiếp theo.

Kết quả mong đợi Người dùng có thể kết nối thành công tài khoản GitHub mà không cần cung cấp mật khẩu trực tiếp cho LivingDocs.

2.2.2 FR2: Repository Selection and Management

Hệ thống phải cho phép người dùng lựa chọn repository cần quản lý sau khi xác thực GitHub thành công.

Mô tả

- Hiển thị danh sách repository mà người dùng có quyền truy cập.
- Cho phép chọn một hoặc nhiều repository.
- Lưu thông tin repository bao gồm tên, owner, branch mặc định và thời gian đồng bộ gần nhất.

Kết quả mong đợi Repository được thêm vào workspace của LivingDocs và sẵn sàng cho quá trình đồng bộ tự động.

2.2.3 FR3: Commit and Pull Request Synchronization

Hệ thống phải đồng bộ thông tin commit và Pull Request từ GitHub với LivingDocs.

Mô tả

- Lấy danh sách commit mới từ branch chính.
- Theo dõi Pull Request được tạo, cập nhật hoặc merge.
- Ghi nhận danh sách file thay đổi trong mỗi Pull Request.
- Liên kết thay đổi mã nguồn với tài liệu liên quan trong LivingDocs.

Kết quả mong đợi LivingDocs luôn phản ánh trạng thái mới nhất của mã nguồn trong repository GitHub.

2.2.4 FR4: CI/CD Integration for Automatic Documentation Update

Hệ thống phải hỗ trợ tích hợp với **GitHub Actions** hoặc công cụ CI/CD tương đương để tự động cập nhật tài liệu.

Mô tả

- Kích hoạt workflow khi có commit hoặc Pull Request mới.
- Build và kiểm tra mã nguồn tự động.
- Gửi thông tin thay đổi đến LivingDocs thông qua API nội bộ.
- Tự động tạo hoặc cập nhật tài liệu liên quan sau khi Pull Request được merge.

Kết quả mong đợi Tài liệu kỹ thuật được cập nhật tự động mà không cần thao tác thủ công từ người dùng.

2.2.5 FR5: Synchronization Status Management

Hệ thống phải quản lý và hiển thị trạng thái của quá trình đồng bộ.

Mô tả

- Theo dõi trạng thái SUCCESS, FAILED hoặc PENDING.
- Lưu thời gian bắt đầu và kết thúc đồng bộ.
- Ghi nhận thông tin lỗi nếu quá trình đồng bộ thất bại.
- Cho phép người dùng thực hiện đồng bộ lại khi cần thiết.

Kết quả mong đợi Người dùng có thể theo dõi và kiểm soát toàn bộ quá trình đồng bộ giữa GitHub và LivingDocs.

2.3 Non-Functional Requirements

2.3.1 NFR1: Security

Hệ thống phải đảm bảo an toàn cho dữ liệu và thông tin xác thực của người dùng.

Yêu cầu

- Access token phải được mã hóa trước khi lưu trữ.
- Chỉ sử dụng kết nối HTTPS/TLS khi giao tiếp với GitHub API.
- Áp dụng cơ chế phân quyền truy cập theo vai trò người dùng.
- Không ghi dữ liệu nhạy cảm vào log hệ thống.

2.3.2 NFR2: Performance

Quá trình đồng bộ phải hoạt động hiệu quả và không ảnh hưởng đáng kể đến hệ thống chính.

Yêu cầu

- Đồng bộ incremental thay vì tải lại toàn bộ repository.
- Ưu tiên xử lý bất đồng bộ đối với các tác vụ phân tích lớn.
- Thời gian phản hồi của API phải đủ nhanh để hỗ trợ trải nghiệm người dùng.

2.3.3 NFR3: Scalability

Hệ thống phải có khả năng mở rộng để phục vụ nhiều repository và nhiều người dùng đồng thời.

Yêu cầu

- Hỗ trợ quản lý nhiều repository trong cùng một tài khoản.
- Cho phép nhiều người dùng sử dụng hệ thống đồng thời.
- Thiết kế dịch vụ theo hướng module hóa để dễ dàng mở rộng trong tương lai.

2.3.4 NFR4: Reliability and Recovery

Hệ thống phải duy trì tính ổn định và có khả năng phục hồi khi xảy ra lỗi kết nối hoặc lỗi dịch vụ bên ngoài.

Yêu cầu

- Tự động thử lại khi GitHub API tạm thời không phản hồi.
- Lưu trạng thái đồng bộ để tiếp tục xử lý sau khi phục hồi.
- Ghi log chi tiết để hỗ trợ giám sát và khắc phục sự cố.

2.3.5 NFR5: Usability

Giao diện người dùng phải trực quan và dễ sử dụng đối với thành viên nhóm phát triển phần mềm.

Yêu cầu

- Các chức năng kết nối GitHub và đồng bộ phải dễ tìm thấy.
- Hiển thị trạng thái đồng bộ rõ ràng bằng biểu tượng hoặc màu sắc.
- Cung cấp thông báo lỗi và hướng dẫn xử lý khi xảy ra sự cố.

2.4 System Constraints

2.4.1 Official GitHub API

Hệ thống phải sử dụng **GitHub REST API** hoặc **GitHub GraphQL API** chính thức để đảm bảo tính tương thích và tuân thủ chính sách của GitHub.

2.4.2 Repository Access Scope

Hệ thống chỉ hỗ trợ các repository:

- Public repository.
- Private repository mà người dùng đã cấp quyền truy cập thông qua OAuth2.

Những repository không có quyền truy cập hợp lệ sẽ không được phép đồng bộ với LivingDocs.

2.4.3 LivingDocs Internal API Integration

Việc trao đổi dữ liệu giữa GitHub Integration và LivingDocs phải được thực hiện thông qua **API nội bộ** của hệ thống.

Ràng buộc

- Chỉ các dịch vụ được xác thực mới được phép gọi API nội bộ.
- Dữ liệu đồng bộ phải tuân theo định dạng chuẩn do LivingDocs quy định.
- Mọi yêu cầu cập nhật tài liệu phải được kiểm tra quyền truy cập trước khi xử lý.

2.5 Conclusion

Phần yêu cầu hệ thống đã xác định các yêu cầu chức năng và phi chức năng quan trọng của hệ thống **LivingDocs**. Các yêu cầu chức năng tập trung vào việc kết nối GitHub bằng OAuth2, quản lý repository, đồng bộ commit và Pull Request, tích hợp CI/CD và quản lý trạng thái đồng bộ. Đồng thời, các yêu cầu phi chức năng đảm bảo hệ thống đáp ứng các tiêu chí về bảo mật, hiệu năng, khả năng mở rộng, độ ổn định và tính dễ sử dụng.

Việc tích hợp GitHub đóng vai trò cốt lõi trong kiến trúc LivingDocs vì nó giúp hệ thống theo dõi chính xác mọi thay đổi của mã nguồn và tự động cập nhật tài liệu tương ứng. Điều này góp phần đảm bảo tài liệu kỹ thuật luôn **cập nhật, nhất quán và phản ánh đúng trạng thái thực tế của hệ thống phần mềm**, từ đó nâng cao chất lượng quản lý dự án và hiệu quả phối hợp của nhóm phát triển.